

IoTPentBench: A Network-Scale Benchmark for LLM-Based IoT Penetration Testing

Anonymous Authors

Paper #XXXX, Anonymized for Double-Blind Review

Abstract—[TODO: Abstract — write last. Lead with the network-scale gap, not the harness.]

Index Terms—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems

1. Introduction

[TODO: Write ¶1: IoT attacks are network-scale; defenders pentest with single-host tooling.]

[TODO: Write ¶2: LLM agents are progressing fast, cite 4-5 recent peers.]

[TODO: Write ¶3: formalize the structural gap. Cite 5-6 benchmarks. Keep it sharp.]

Contributions. We bridge this gap with:

- **IoTPentBench** — the first reproducible benchmark for LLM pentest agents at *network scale*: 5 architectural patterns, 7 scenarios, 8 protocols, 87 vulnerabilities deterministically injected via Ansible, with ground truth mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS.
- **MESHSCOUT** — a network-native harness with *discovery* → *per-device parallel triage* → *per-vulnerability verification* → *network-level report*, designed to keep LLM context tractable at /24 scale.
- **Evidence-level scoring (L1/L2/L3)** — a severity-weighted evaluator that distinguishes detection, exploitation, and data exfiltration, beyond the binary flag-capture of single-host benchmarks.
- **Empirical study** — using a single shared LLM (MiniMax-M2.7) to isolate architectural contributions, we compare MESHSCOUT against three categories of baselines: a non-LLM scanner (Nmap NSE), single-agent LLM (PentestGPT), and multi-agent LLM systems with and without graph guidance (CAI in two scope variants, VulnBot with task-graph). We additionally report two ablations of our pipeline that isolate the individual contributions of the infrastructure-graph phase and the multi-hop intrusion phase.

[TODO: Write ¶5: one teaser sentence per RQ. e.g. "Best LLM reaches XX% recall at /24 scale; scanners reach YY%; but only ZZ% of LLM findings achieve L2+."]

Paper organization. Section 2 surveys LLM pentest agents and benchmarks. Section 3 formalizes the threat model. Section 4 presents IoTPentBench. Section 5 describes the harness architecture. Section 6 and Section 7 present experiments and results. Section 8 and Section 9 discuss limitations and ethics. Artifacts are released at ANONYMIZED_URL.

2. Background and Related Work

2.1. LLM-based penetration testing agents

Single-agent origins. Deng *et al.* proposed PENTEST-GPT [1], the first published LLM-driven penetration testing agent. PENTESTGPT couples a single ChatGPT-class model with a human-in-the-loop interface that translates natural-language guidance into shell commands, and was evaluated on 182 hand-selected sub-tasks drawn from HackTheBox and VulnHub targets. The work established three findings that recur throughout the subsequent literature: LLMs produce plausible reconnaissance plans, they excel on text-rich web vulnerabilities (SQLi, XSS, IDOR), and they fail on long-context tasks where state must be maintained across many command invocations.

Multi-agent decomposition. Later systems address the context limitation through role specialization. VULNBOT [2] decomposes the task into three phase-specific agents (*reconnaissance*, *scanning*, *exploitation*) coordinated by a Penetration Task Graph (PTG); on the AUTOPEN-BENCH corpus [3] its LLAMA3.1-405B variant reaches a completion rate of 30.3% versus 9.1% for the same model used single-agent and 21.2% for GPT-4o. AUTOPEN-TESTER [4] uses five cooperating agents (summarizer, strategy analyzer, command generator, results verifier, reporter) with Retrieval-Augmented Generation and a Pentest Tree data structure, reporting a 27% gain in sub-task completion and 39.5% more vulnerability coverage over PENTESTGPT on HackTheBox plus a custom set of OWASP-Top-10 machines. PENTESTAGENT [5] likewise builds on a multi-stage pipeline with RAG, published at ACM ASIA CCS'25.

Learning-based and long-running variants. PENTEST-R1 [6] applies two-stage reinforcement learning to the reasoning traces of a mid-scale model, and XOFFENSE [7] fine-tunes a Qwen3-32B with offensive knowledge corpora, reporting 79% sub-task completion on CTF-style targets. A complementary strand addresses session duration: CHAP [8] introduces a

context-relay mechanism that transfers distilled protocols between successive agent instances during long-running engagements, and AWE [9] adds persistent memory and browser-backed verification to web-application pentest. CHECKMATE [10] couples LLM agents with classical planning to obtain deterministic action selection, improving benchmark success by over 20 percentage points.

Production-grade frameworks. CAI [11], developed by the PENTESTGPT authors, generalizes the single-agent prototype into a bug-bounty-oriented framework with modular tool support and a public leaderboard; on CAIBench’s CTF subset it consistently outperforms prior open-source systems.

Gap. Despite this trajectory, every system in this cluster is evaluated on a *single-host* target unit: HackTheBox machines, VulnHub VMs, AUTOPENBENCH challenges, or CTF containers. Success is reported either as flag capture or as sub-task completion within one host. None evaluates at the scale of a deployed IP network containing heterogeneous devices and IoT-specific application protocols.

2.2. Penetration testing benchmarks

Academic corpora. AUTOPENBENCH [3] is the current academic reference, consisting of VulnHub-derived single-VM challenges with scripted flag checkers; it is used as the canonical evaluation by VULNBOT and PENTEST-R1. PENTESTGPT reported on a hand-curated set of 182 sub-tasks drawn from HackTheBox and VulnHub. More recently, CAIBENCH [12] consolidates the field into a meta-benchmark spanning Jeopardy-style CTFs, attack-and-defense exercises, cyber-range scenarios, and knowledge/privacy assessments, reporting that state-of-the-art models saturate around 70% on static knowledge tests but collapse to 20–40% on multi-step adversarial tasks. Within CAIBENCH, the RCTF2 subset contributes 27 challenges targeting robotics and cyber-physical systems — the first inclusion of CPS in an LLM-pentest benchmark — yet each challenge remains an isolated CTF-style target rather than a deployed network.

Training-oriented lab infrastructures. LUDUS [13] and the GOAD [14] Active-Directory scenario deliver reproducible Proxmox-based labs with Ansible-driven deployment and idempotent configuration. Methodologically these projects are the closest precedent to our infrastructure, but they are training tools for human operators: they do not ship a ground-truth schema, do not define a scoring procedure, and have not been used to evaluate LLM agents in the published literature.

Gap. No benchmark in the published literature provides all of (i) network-scale evaluation on a deployed multi-device IP range, (ii) IoT-specific application protocols (MQTT, LoRaWAN, Modbus, CoAP), (iii) per-vulnerability ground truth usable for automated scoring, and (iv) evidence-level granularity beyond binary flag capture. IoTPentBench is designed to fill exactly this intersection.

2.3. IoT security methodology

We ground scenario design and ground-truth taxonomy in three widely adopted frameworks. The OWASP IoT Top 10 [15] structures our vulnerability categories and ensures coverage of the attack surfaces most commonly observed in IoT deployments (hardcoded credentials, insecure network services, insecure ecosystem interfaces, insufficient update mechanisms). The MITRE ATT&CK for ICS [16] matrix provides a tactics vocabulary for OT-inclusive scenarios; each injected vulnerability in IoTPentBench carries a MITRE-ICS tactic annotation where applicable. Finally, NIST SP 800-82 Rev. 3 [17] informs our segmentation and safety conventions, in particular the IT/OT separation underpinning architectural pattern P3 (§4.2).

3. Threat Model and Problem Definition

Network-scale pentest. We define *network-scale penetration testing* as the task of: (i) discovering all live hosts in a target IP range, (ii) enumerating services and identifying vulnerabilities on each host, (iii) producing evidence of exploitability where feasible, and (iv) aggregating findings at the network level. The *unit of evaluation* is the network, not an individual host.

Attacker model. The attacker holds network-layer access to the target subnet (post-ingress, pre-compromise). They hold no prior credentials, no privileged vantage, and no knowledge of deployed software beyond what is observable through scans. The attacker has access to standard offensive tooling (Nmap, MQTT clients, SSH, ...) and an LLM with tool-calling. Time and budget are bounded.

Goals.

- 1) Enumerate all devices reachable from the entry point.
- 2) For each device, identify vulnerabilities from misconfiguration, weak credentials, information disclosure, insecure protocols, and known CVEs.
- 3) For each finding, attempt to produce *evidence* at one of three levels: L1 detection (port open, banner leak), L2 exploitation (authenticated session, protocol handshake), L3 data exfiltration (configuration files, credentials, sensor data).
- 4) Report findings at the network level (severity-ranked, deduplicated).

Scope and non-goals. We evaluate at the *IP/application layer* on deployed IoT networks. We do *not* evaluate: (a) physical/side-channel attacks on hardware, (b) RF-layer attacks requiring SDR captures, (c) firmware extraction from physical devices, (d) post-exploitation persistence or impact, (e) insider threats or supply-chain compromise. Hardware-assisted attacks are left as future work (§10).

TABLE 1. COMPARISON WITH PRIOR LLM PENETRATION TESTING SYSTEMS AND BENCHMARKS. “SCOPE” IS THE UNIT OF EVALUATION; “NETWORK-SCALE” MEANS EVALUATION OCCURS ON A MULTI-DEVICE IP NETWORK RATHER THAN ISOLATED TARGETS.

System	Scope	Protocols	Architectures	IoT-specific	Scoring	Ground truth	Reproducible
PentestGPT [1]	single-host	Web/OS	n/a	no	sub-task	hand-curated	partial
VulnBot [2]	single-host	Web/OS	n/a	no	task-graph	AUTOPENBENCH	yes
AutoPentester [4]	single-host	Web/OS	n/a	no	sub-task	HTB + custom	yes
PentestAgent [5]	single-host	Web/OS	n/a	no	composite	custom	partial
Pentest-R1 [6]	single-host	Web/OS	n/a	no	sub-task	AUTOPENBENCH	partial
xOffense [7]	single-host	Web/CTF	n/a	no	sub-task	CTF	partial
CAI [11]	single-host	Web/CTF	n/a	no	CTF	CAIBench	yes
CAIBench / RCTF2 [12]	single-host	mixed	n/a	27 CPS	CTF	yes	yes
Ludus + GOAD [13], [14]	multi-host	AD / Win	AD only	no	(training)	no	yes
IoTPentBench (ours)	network	MQTT, Modbus, LoRa, ...	5 patterns	yes	L1/L2/L3 + wt.	per-vuln YAML	yes

4. IoTPentBench: A Network-Scale Benchmark

4.1. Design principles

IoTPentBench is built around four principles derived from the gap analysis in §2:

- 1) **Reproducibility.** The entire infrastructure deploys from Ansible playbooks against a Proxmox cluster; every vulnerability is injected idempotently with no manual post-deployment steps.
- 2) **Architectural diversity.** Five network patterns (§1) with distinct attacker reachability graphs enforced by OpenWrt firewall rules, not just by service labels.
- 3) **Protocol coverage.** Each scenario mixes multiple IoT and OT protocols (MQTT, Modbus, LoRaWAN gateways, CoAP, SNMP, plus standard SSH/HTTP/FTP/Telnet) to exercise protocol-specific reasoning.
- 4) **Ground-truth injectability.** Every injected vulnerability is documented in YAML with device, IP, role, severity, category, OWASP IoT and MITRE ICS mappings, optional CVE, indicators (expected detection strings), and a verification command.

4.2. Five architectural patterns

[TODO: Write §4.2 prose — for each of the 5 patterns: (i) one sentence naming the pattern, (ii) the real-world IoT deployment it models, (iii) the firewall reachability invariant.]

Concretely:

- 1) **Flat.** All devices are directly reachable from the entry point; no pivot is required.
- 2) **Gateway-centric.** A single IoT gateway mediates all access to back-end services; inter-service traffic is blocked without gateway compromise.
- 3) **Segmented IT/OT.** Three zones (admin/IT/OT) with access-control lists; the OT zone (Modbus PLCs, HMI) is reachable only via a jump host.

TABLE 2. VULNERABILITY CATEGORIES AND SEVERITY DISTRIBUTION ACROSS IOTPENBENCH (87 INJECTED INSTANCES).

Category	Count	CRIT	HIGH	MED/LOW
misconfiguration	17	3	8	6
data_exposure	12	0	7	5
default_credentials	11	6	5	0
info_disclosure	10	0	0	10
no_authentication	9	2	4	3
insecure_update	4	0	0	4
weak_crypto	3	0	0	3
missing_header	3	0	0	3
known_cve	3	0	3	0
code_injection	3	2	1	0
privilege_escalation	2	0	2	0
Total	87	13	30	44

- 4) **Hub-centric star.** A central home-automation hub is the only device that can reach MQTT broker, DB, and camera; peripherals cannot talk to each other directly.
- 5) **Edge-cloud pivot.** Edge subnet and cloud subnet are separated; only the edge gateway bridges them, making cloud-side services unreachable without edge compromise.

4.3. Deterministic vulnerability injection

Vulnerabilities are injected by a single Ansible role (04_inject_vulns.yml) parameterized by scenario_id. Each scenario YAML lists router_vulns (OpenWrt-level: Telnet, WAN admin, FTP) and per-service role definitions that trigger role-specific injections: *e.g.*, the mqtt_broker role sets allow_anonymous true and binds on 0.0.0.0:1883; the ssh_server role installs weak credentials (admin:admin) and enables PasswordAuthentication; the db_server role starts MariaDB with a passwordless root. Category distribution is shown in §2.

4.4. Ground-truth schema

Each scenario ships a YAML file describing every injected vulnerability. An entry example (from scenario_1.yaml):

*Fig. 1 placeholder: five topology diagrams side-by-side.
(Flat) — (Gateway-centric) — (Segmented IT/OT) — (Hub-star) — (Edge-cloud)*

Figure 1. The five architectural patterns in IoTpentBench. Each pattern is enforced by OpenWrt firewall rules that constrain the attacker’s reachability from the entry point, not by service labels alone.

```
- id: V1
  device: s1-mqtt
  ip: "192.168.100.11"
  role: mqtt_broker
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  cve: null
  indicators:
    - "allow_anonymous true"
    - "Anonymous clients connected"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 2. Ground-truth entry example.

4.5. Evaluator

The evaluator (§5.6 integrates results into Phase 5) matches each LLM finding against ground truth in four passes, in order: (1) exact CVE match; (2) (ip, vuln_type); (3) (ip, category); (4) (ip, severity). Severity weights are $w = \{\text{critical}=4, \text{high}=3, \text{medium}=2, \text{low}=1\}$; loose category matches apply $0.5\times$, severity mismatches $0.75\times$. The composite score is the weighted True-Positive sum normalized by $\sum_{g \in \text{GT}} w(g)$.

Evidence levels. Beyond match accuracy, the evaluator tracks, per True Positive, the deepest *evidence level* achieved by the harness: L1 (detection — banner or port), L2 (exploitation — authenticated action), L3 (data exfiltration — sensitive content retrieved). We report $L_k\text{-coverage} = |\{t \in \text{TP} : \text{level}(t) \geq k\}|/|\text{TP}|$. This metric cannot be computed on single-host binary-flag benchmarks and captures a dimension of pentest quality that existing LLM-pentest evaluations conflate.

5. MESHSCOUT: A Network-Native LLM Pentest Harness

5.1. Overview

The harness decomposes network pentest into five phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent that must hold the full network state, we spawn dedicated micro-agents per target with constrained tool sets, then aggregate deterministically. This keeps LLM context bounded at $O(1)$ per device regardless of network size.

5.2. Phase 1: Topology prior

Phase 1 loads a graph model of the target network as a directed graph $G = (V, E)$ where V are devices and E are links labeled by protocol. Two modes are supported: (i) *scenario mode* loads the predefined topology from the benchmark YAML; (ii) *discovery mode* starts from an empty graph and instructs Phase 2 to populate V via network scans. The output is a markdown deliverable enumerating attack surface and preliminary pivot candidates (betweenness-centrality ranked).

5.3. Phase 2: Full-network discovery

Phase 2 executes active reconnaissance across the target /24: `nmap_discovery` (host discovery), `nmap` (service/version), `arp_scan` (L2 + vendor), and protocol-specific probes (`mqtt_listen` subscribing to # on discovered brokers, `ssh_audit` against SSH servers, `curl_headers` on HTTP). Unlike single-host harnesses, the agent is not given a target IP — it must derive one from the CIDR scope.

5.4. Phase 3: Per-device parallel triage

Phase 3 runs in three sub-phases:

- 1) **Deterministic scanner pass.** Ansible-driven subprocess wrappers (`ssh_audit`, `curl_headers`, `mqtt_listen`, `modbus_scan`, ...) run in parallel per discovered device and dump structured scan results to disk.
- 2) **Per-device LLM triage.** For each device, a dedicated LLM agent receives only that device’s Phase 2+3a outputs and has access to a minimal tool set: `http_get` (follow-up file retrieval) and `cve_search` (CPE-to-CVE lookup via NVD). The agent produces a JSON vuln list scoped to that device.
- 3) **Deterministic merge.** Per-device outputs are concatenated, canonicalized through the taxonomy (§5.7), and deduplicated via severity-aware rules.

This decomposition is essential for network scale: a monolithic agent on a 15-device scenario would need to fit $\sim 30\text{--}80\text{k}$ tokens of scan output into a single context and reason about cross-device relationships simultaneously. Per-device micro-agents flatten this to a constant size, at the cost of losing some cross-device inference capability, which is recovered in Phase 5.

Fig. 2 placeholder: 5-phase pipeline diagram.
Phase 1 (topology prior) → Phase 2 (discovery) → Phase 3 (per-device triage) → Phase 4 (per-vuln verification) → Phase 5 (network report).

Figure 3. MESHSOUT pipeline. Solid arrows are deliverables; dashed boxes indicate LLM agent calls; Phase 3 and Phase 4 spawn micro-agents in parallel (§5.4–§5.5).

Fig. 3 placeholder: Phase 3 parallelization — n devices → n LLM agents in parallel → deterministic merge.

Figure 4. Per-device parallelization. Each micro-agent receives only its device’s scan output, avoiding the context explosion of a monolithic network-level agent.

5.5. Phase 4: Per-vulnerability verification with evidence levels

Phase 4 spawns one micro-agent per Phase 3 finding (in parallel, with a concurrency cap). Each micro-agent receives the finding (IP, type, port, details) and full operational tool access: `ssh_login`, `mysql_query`, `mqtt_listen`, `http_get`, `ftp_list`, `modbus_scan`, `telnet_connect`. The agent is prompted to produce the deepest evidence level it can (L1–L3, §4.5) within a bounded turn budget. Outputs include: `status` (CONFIRMED, NOT_EXPLOITABLE, ERROR), `evidence_level`, command executed, `stdout` excerpts, and `data_extracted` (when L3).

Findings in CONFIRMED state from Phase 3 override Phase 4 ERROR/FAILED statuses (rationale: Phase 3 is hypothesis generation with directly-observed indicators; Phase 4 crashes should not erase directly-observed findings).

5.6. Phase 5: Network-level report

Phase 5 consumes all prior deliverables and produces a network-scoped report with: device inventory, per-device vulnerabilities with severity, attack surface summary, and actionable recommendations ordered by severity $\times$ exposure. This is the only phase with cross-device reasoning, deliberately placed at the end to avoid bloating earlier phases.

5.7. Taxonomy and tool abstraction

A single module resolves LLM-produced vulnerability type strings into a canonical taxonomy of 11 categories (§2), filtering noise (e.g., meta-observations) and CONFIG-only findings. Tools are defined declaratively as YAML (schema: name, description, JSON-schema parameters, subprocess command and template), so adding a new protocol probe requires no Python code.

5.8. Complexity and cost envelope

For a network with n devices and an average of v findings per device, the harness makes $O(1 + n + nv)$ LLM

calls (Phases 1, 3b, 4 respectively), bounded per call by fixed turn budgets (50/50/10/10/20 for phases 1–5). Total cost scales linearly with network size, not combinatorially.

6. Experimental Setup

Infrastructure. All scenarios run on a single Proxmox VE host. Networks are Linux bridges (`vmbri1`, `192.168.100.0/24`) with an OpenWrt router as gateway. Per-scenario deployment and teardown are automated via Ansible and complete in under 10 minutes per scenario.

LLM model — fairness lock. To isolate *architectural* contributions from model-specific advantages, all LLM-driven systems compared in this work — our pipeline (and its ablations) and all LLM baselines — run on the *same* general-purpose model: **MiniMax-M2.7** [18]. We deliberately avoid a multi-model matrix: a fair comparison of architectures requires the model variable to be held constant. CAI’s specialised `alias1` model (security-fine-tuned, pay-walled) is intentionally not used — this study addresses the architecture axis, not the model fine-tuning axis. The choice of MiniMax-M2.7 also keeps the combined API cost at zero under our research subscription, enabling the full $k=3$ -run variance estimation across all systems and scenarios without budget trade-offs.

Baselines. We compare our pipeline against three categories of systems, each occupying a distinct architectural slot:

- **Single-agent LLM** — PENTESTGPT [1] in autonomous mode, the canonical USENIX’24 reference.
- **Multi-agent LLM (no infrastructure graph)** — CAI [11]. We test two variants: *A1* per-IP (one independent CAI session per ground-truth IP, no shared state) and *B* CIDR-scope (a single CAI session given the full /24, allowed to discover hosts and pivot via `LinuxCmd` primitives). Both variants receive the same total turn budget as our pipeline.
- **Multi-agent LLM with task-graph** — VULNBOT [2], the most structurally similar competitor. VulnBot’s Penetration Task Graph (PTG) is a directed acyclic graph of *tasks* (action dependencies), to be contrasted with our infrastructure graph of *devices* (firewall-enforced reachability). Both are “graph-based”, but encode orthogonal abstractions.
- **Non-LLM scanner** — Nmap with all NSE scripts (`--script=default,vuln,auth`) against the same target CIDR. Scanner outputs are converted to the evaluator’s JSON schema.

Internal ablations. Two ablations isolate the contribution of our two key design choices: (i) MESHSCOUT-w/o-Phase 1 disables graph analysis (--phases 2 3 4 5 6), testing the value of infrastructure-graph guidance; (ii) MESHSCOUT-w/o-Phase 5 disables the intrusion phase (--phases 1 2 3 4 6), testing the value of structured multi-hop lateral movement.

Variance. Each (system, scenario) cell is run $k=3$ times. We report mean, standard deviation, and 95% bootstrap confidence intervals. Pairwise differences (our pipeline vs each baseline) are tested with Mann-Whitney U on F1 and weighted Score per scenario. This protocol exceeds the single-run reporting of PentestGPT [1], AutoPentester [4], AUTOPENBENCH [3], and CAI [11], and matches the multi-run averaging of VulnBot [2] (5-experiment) and Pentest-R1 [6] (pass@3).

Metrics. We report five families of metrics, chosen to (i) align with prior work’s expected vocabulary and (ii) surface architectural differences invisible to host-level benchmarks:

- *Task completion* — Recall against the ground-truth vulnerability set, the network-scale analogue of the sub-task / task completion rate reported by PentestGPT, VulnBot, AutoPentester, Pentest-R1, and xOffense.
- *Quality* — Precision, F1, and weighted Score (CVSS-severity-weighted Recall, §4.5). Score plays the role of AutoPentester’s “vulnerability coverage” but with severity weights.
- *Multi-Hop Reach* — MHR_k for $k \in \{1, 2, 3\}$, the fraction of ground-truth vulnerabilities at $\text{hop_depth} \geq k$ that are reached. No prior LLM-pentest benchmark measures network depth explicitly.
- *Evidence depth* — L_1 (detection), L_2 (exploitation), L_3 (exfiltration) coverage; absent from binary flag-capture benchmarks.
- *Cost envelope* — prompt+completion tokens and monetary cost (USD) from provider billing APIs, plus wall-clock time per scenario, matching CAI [11], AutoPentester, and Pentest-R1.

Compute and budget. All runs consume a combined API budget under \$NNN, bounded by per-phase turn caps and the concurrency limit in Phase 4. **[TODO: Insert exact budget after all runs complete (freeze 2 May).]**

7. Evaluation

7.1. Overall performance (RQ1)

[TODO: Write §7.1 prose — headline finding: MESHSCOUT achieves $MHR_2 \approx XX\%$, while all baselines collapse to near-zero on multi-hop vulnerabilities. F1 gap to nearest baseline (VulnBot) is ZZ percentage points. Variance across 3 runs stays under YY%.]

Fig. 4 placeholder: bar chart, F1 and MHR_2 for MESHSCOUT vs CAI (A/B), VulnBot, PentestGPT, Nmap NSE.

Figure 5. Baseline comparison. Scanners capture known CVEs but miss configuration vulnerabilities. Single-host LLM agents (PentestGPT, CAI A1) cannot pivot across firewall boundaries. CAI B (given the full CIDR) and VulnBot (multi-agent + task-graph) approach our pipeline on direct findings but collapse on multi-hop reach (MHR_2 , MHR_3).

Fig. 5 placeholder: heatmap scenario $\times$ model, cell = weighted Score.

Figure 6. Per-scenario Score heatmap. **[TODO: Note which pattern consistently breaks LLMs.]**

7.2. Network-scale vs baselines (RQ4)

[TODO: Write §7.2 — explicit narrative: (i) scanners win on CVE category, (ii) LLMs win on misconfig/default-creds, (iii) single-host LLMs adapted to network fall behind native harness on F1 by ZZ%.]

7.3. Per-architecture analysis (RQ3)

[TODO: Write §7.3 — pattern-by-pattern commentary. Expected finding: gateway-centric and hub-star degrade most, because they require pivot reasoning. Edge-cloud may also degrade on cloud-side service discovery.]

7.4. Evidence-level depth (RQ2)

[TODO: Write §7.4 — expected: detection (L_1) near-parity with scanners, but L_2/L_3 is LLM-only. This is the strongest argument for LLM agents in this context.]

7.5. Per-protocol analysis

[TODO: Write §7.5 — 2 paragraphs. Expected: LLMs excel on MQTT/SSH/HTTP (text-rich, well-represented in training), fall short on Modbus/LoRaWAN (binary, sparse training data). Report numbers per protocol.]

7.6. Failure modes and cost/time envelope

[TODO: Write §7.6 — categorize failure modes: (a) hallucinated findings on devices with no services, (b) missed vulnerabilities on binary protocols, (c) over-reporting info-disclosure without severity calibration. Brief quantitative table.]

8. Discussion and Limitations

Where LLMs help, where they don’t. IoTPentBench surfaces an empirical pattern: LLM agents reach high recall on misconfigurations, default credentials, and information disclosure — findings that are textually expressible and well

TABLE 3. MAIN RESULTS — TASK COMPLETION (RECALL), PRECISION, F1, WEIGHTED SCORE, MULTI-HOP REACH (MHR_k), TOKENS, AND USD COST; AVERAGED OVER 7 SCENARIOS $\times$ 3 RUNS. ALL LLM SYSTEMS USE MINIMAX-M2.7 (CF. §6); NMAP NSE IS THE NON-LLM LOWER BOUND. *Task Completion* IS THE NETWORK-SCALE ANALOGUE OF THE SUB-TASK COMPLETION RATE REPORTED BY PENTESTGPT, VULNBOT, AUTOPENTESTER, AND XOFFENSE (§7.1). MHR_k MEASURES THE FRACTION OF GROUND-TRUTH VULNERABILITIES AT $HOP_DEPTH \geq k$ THAT WERE REACHED; “–” INDICATES NO GT ENTRIES AT THAT DEPTH. CELLS PRE-FILLED WITH 0.00 ARE ARCHITECTURALLY IMPOSSIBLE: SINGLE-HOST LLMs AND SCANNERS CANNOT PIVOT ACROSS FIREWALL BOUNDARIES BY CONSTRUCTION.

System	Quality				Multi-Hop Reach			Cost	
	TC	Prec.	F1	Score	MHR_1	MHR_2	MHR_3	Tokens (M)	USD
MESHSCOUT (full)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 1 (no infra graph)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 5 (no intrusion)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	[TODO:]	[TODO:]
CAI [11] (Variant A1, per-IP)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	[TODO:]	[TODO:]
CAI [11] (Variant B, CIDR scope)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
VulnBot [2] (multi-agent + PTG)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
PentestGPT [1] (single-agent)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	[TODO:]	[TODO:]
Nmap NSE (non-LLM scanner)	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]	0.00	0.00	–	–

TC = Task Completion (Recall against GT vuln set). Reported across $k=3$ runs with 95% bootstrap CI and pairwise Mann–Whitney U vs each baseline; this protocol exceeds the single-run reporting of PentestGPT, AutoPentester, AUTOPENBENCH, and CAI, and matches the multi-run averaging of VulnBot and Pentest-R1.

TABLE 4. EVIDENCE-LEVEL COVERAGE. L_k IS THE FRACTION OF TRUE POSITIVES REACHING LEVEL k OR DEEPER. ABSENT FROM BINARY-FLAG BENCHMARKS.

System	L_1 (detect)	L_2 (exploit)	L_3 (exfiltrate)
MESHSCOUT (full)	[TODO:]	[TODO:]	[TODO:]
MESHSCOUT w/o Phase 5	[TODO:]	[TODO:]	0.00
CAI (Variant A1) [11]	[TODO:]	[TODO:]	0.00
CAI (Variant B) [11]	[TODO:]	[TODO:]	[TODO:]
VulnBot [2]	[TODO:]	[TODO:]	[TODO:]
PentestGPT [1]	[TODO:]	[TODO:]	0.00
Nmap NSE	–	–	–

Fig. 6 placeholder: Pareto plot, USD cost vs weighted Score, one point per model.

Figure 7. Cost–performance Pareto. [TODO: Annotate which model dominates on each axis and the efficient frontier.]

represented in their training corpora. They consistently underperform on binary industrial protocols (Modbus, CoAP, LoRaWAN) and on cross-device inferences that require holding an aggregated network view. This suggests a hybrid deployment model: LLM agents for configuration auditing, traditional scanners for CVE coverage, and human operators for the OT/ICS layer.

Implications for agentic systems. The harness’s per-device and per-vulnerability micro-agent structure is one answer to context scaling, but it cedes cross-device inference. Designs that keep a global aggregator agent in the loop without paying the full network-context cost are an open research direction.

Limitations. (i) *Simulated infrastructure*. IoTPentBench scenarios emulate IoT devices on Linux VMs; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) we do not cover. (ii) *Evaluator leniency*. Bonus categories are accepted as

non-FP even without ground-truth matches, which inflates precision. (iii) *Model variance*. With $k=3$ runs per configuration, confidence intervals on F1 remain wide for some scenarios. (iv) *Prompt sensitivity*. We fix a single prompt family across models; per-model prompt tuning would likely raise scores. (v) *Static architectures*. The five patterns are fixed; real deployments evolve over time and include devices not represented here (RFID badge readers, industrial PLCs, medical devices).

9. Ethical Considerations

Isolation. All experiments run on an isolated Proxmox cluster with a dedicated bridge (vbr1) and no route to external networks; the OpenWrt gateway’s WAN interface is firewall-isolated. No live hosts, production services, or third-party infrastructure are probed.

Responsible disclosure. IoTPentBench vulnerabilities are deterministically injected into our own lab VMs via Ansible. No 0-days are introduced; all categories (misconfig, weak credentials, Terrapin CVE-2023-48795, outdated Dropbear) correspond to patterns already publicly documented. No responsible-disclosure process is triggered.

Misuse risk of released artifact. The benchmark artifact (Ansible playbooks, ground truth, evaluator) does not embed exploitation payloads for unknown vulnerabilities, and the injected weaknesses are common misconfigurations documented in OWASP IoT Top 10. The harness uses only standard pentest tools already in widespread use. We judge the net societal benefit — reproducible evaluation of LLM pentest agents, and a concrete evaluation surface for future defenders — to exceed the incremental misuse risk.

IRB / human subjects. No human subjects are involved. No personal data is collected or processed.

LLM costs and environmental footprint. All reported experiments together consume an estimated [TODO: X] kWh and cost under [TODO: USD Y] in API billing.

10. Conclusion

We presented IoTPentBench, the first reproducible benchmark for evaluating LLM penetration testing agents at *network scale* on IoT infrastructure, and MESHSCOUT, a network-native multi-agent harness whose infrastructure-graph guidance and dedicated lateral movement phase keep LLM context tractable while explicitly modelling firewall-enforced reachability. Holding the LLM constant (MiniMax-M2.7) across our pipeline, two ablations, and four external baselines (CAI in two scope variants, VulnBot, PentestGPT, Nmap NSE), we isolate the *architectural* contribution: MESHSCOUT outperforms single-host LLM agents on multi-hop reach (MHR_2 , MHR_3) by a margin that collapses near zero for every baseline by construction, and demonstrates that the L1–L3 evidence spectrum exposes architectural differences that binary flag-capture benchmarks conflate.

Future work. (i) *Multi-model robustness* — extending the architectural comparison to recent frontier models (Claude Sonnet 4.6, GPT-5.5, Gemini 3.1 Pro, DeepSeek V4, Qwen 3.6) to confirm model-independence of the architectural advantage; (ii) *Real hardware* — extending to embedded Zigbee, LoRaWAN, and sub-GHz attack surfaces via SDR tooling; (iii) *Defender’s side* — using IoTPentBench as a benchmark for LLM-assisted IoT defense (detection, patching recommendation).

All code, scenarios, ground truth, and run logs are released at ANONYMIZED_URL under an open license.

References

- [1] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *33rd USENIX Security Symposium*, 2024. [Online]. Available: <https://pentestgpt.com/paper.html>
- [2] H. He *et al.*, “VulnBot: Autonomous penetration testing for a multi-agent collaborative framework,” *arXiv preprint arXiv:2501.13411*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.13411>
- [3] Anonymous, “Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements,” *arXiv preprint arXiv:2410.17141*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.17141>
- [4] Y. Ginige *et al.*, “AutoPentester: An LLM agent-based framework for automated pentesting,” *arXiv preprint arXiv:2510.05605*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.05605>
- [5] X. Shen *et al.*, “PentestAgent: Incorporating LLM agents to automated penetration testing,” in *Proc. 20th ACM Asia Conference on Computer and Communications Security*, 2025.
- [6] Anonymous, “Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning,” *arXiv preprint arXiv:2508.07382*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.07382>
- [7] —, “xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi-agent systems,” *arXiv preprint arXiv:2509.13021*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.13021>
- [8] —, “Context relay for long-running penetration-testing agents,” 2026. [Online]. Available: <https://www.ndss-symposium.org/wp-content/uploads/lastx2026-42.pdf>
- [9] —, “AWE: A memory-augmented multi-agent framework for autonomous web penetration testing,” in *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X)*, 2026.
- [10] —, “Automated penetration testing with LLM agents and classical planning,” *arXiv preprint arXiv:2512.11143*, 2025. [Online]. Available: <https://arxiv.org/abs/2512.11143>
- [11] V. Mayoral-Vilches *et al.*, “CAI: An open, bug bounty-ready cybersecurity AI,” *arXiv preprint arXiv:2504.06017*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.06017>
- [12] —, “CAIBench: A meta-benchmark for evaluating cybersecurity AI agents,” *arXiv preprint arXiv:2510.24317*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.24317>
- [13] L. Cloud, “Ludus: Cyber range automation on Proxmox,” <https://ludus.cloud>, accessed Apr. 2026.
- [14] M3, “GOAD: Game of active directory,” <https://github.com/Orange-Cyberdefense/GOAD>, accessed Apr. 2026.
- [15] OWASP Foundation, “OWASP Internet of Things Top 10,” <https://owasp.org/www-project-internet-of-things-top-10/>, 2024.
- [16] MITRE Corporation, “MITRE ATT&CK for ICS,” <https://attack.mitre.org/matrices/ics/>, 2024.
- [17] K. Stouffer *et al.*, “Guide to Operational Technology Security,” National Institute of Standards and Technology, Tech. Rep. SP 800-82 Rev. 3, 2023.
- [18] MiniMax, “MiniMax-M2.7: A general-purpose large language model,” <https://www.minimax.io/models/text/m27>, 2026, accessed Apr. 2026.

LLM Usage Statement

This paper involves Large Language Models in two distinct roles.

LLM as subject of study. The experimental work evaluates the penetration testing capabilities of LLM agents using MiniMax-M2.7 [18] as the shared model across MESHSCOUT and all baselines (CAI, VulnBot, PentestGPT) to isolate the architectural contribution from the model variable (see §6). All inference calls were made through MiniMax’s official API under standard terms of service. No model weights were fine-tuned; all systems were used out-of-the-box with identical tool schemas and prompt templates per system. Prompt templates, full conversation transcripts, and per-phase token counts are released as part of the artifact.

LLM as writing aid. During manuscript preparation, the authors used current frontier LLMs (Anthropic Claude Opus 4.7, OpenAI GPT-5.5) for: (i) drafting candidate section outlines subsequently edited by the authors; (ii) copy-editing for grammar, clarity, and concision; (iii) consistency checks on terminology and notation. No section was written end-to-end by an LLM. All claims, experimental results, related-work positioning, and scientific interpretations are the authors’ own. Every LLM-suggested citation was verified by retrieving the cited work.